

Autoencoder-Based Open-Set Detection of Zero-Day Network Intrusion

Rameen Sohail — 01-134232-267

Department of Computer Science, Bahria University, Islamabad, Pakistan

Course: Advanced Deep Learning (CS-601) | Submitted to: Miss Afrah

ABSTRACT

Network intrusion detection systems built on standard deep learning classifiers share one critical flaw. When a completely new, never-before-seen attack arrives, the model does not raise an alert. Instead, it confidently labels the attack as normal traffic and lets it pass through undetected [5], [7]. This happens because all standard classifiers use a SoftMax output layer that forces 100 percent probability across known classes, making the model mathematically incapable of expressing uncertainty [6], [7]. This paper addresses that gap by proposing the Autoencoder-Based Open-Set Detection (ABOSD) framework, which combines an LSTM classifier for known attack classification with an autoencoder that flags unfamiliar traffic as suspicious based on reconstruction error [1], [6]. The system was trained on CICIDS2017 (2.8 million records) and evaluated on two completely separate datasets, namely UNSW-NB15 and NSL-KDD, with no retraining involved. ABOSD detected 99.85% of unknown attacks from UNSW-NB15 and 90.20% from NSL-KDD, with only a 2.00% false alarm rate on known traffic, confirming that reconstruction error is a reliable and scalable signal for detecting zero-day threats at scale.

I. INTRODUCTION

Cybersecurity has become a critical concern as modern organizations increasingly rely on digital networks for daily operations [1]. As cloud computing, online services, and interconnected systems continue to expand, the attack surface available to cybercriminals grows with them [2]. Detecting malicious activity in network traffic has therefore become a fundamental requirement for maintaining secure computing environments [1]. One of the most widely used tools for this purpose is the Network Intrusion Detection System (NIDS), which continuously monitors network traffic and attempts to identify patterns associated with malicious behavior [2].

Deep learning has significantly improved intrusion detection over the past several years [9], [10]. Models such as Long Short-Term Memory networks (LSTM), Convolutional Neural Networks (CNN), and various hybrid architectures have been applied to network traffic datasets, often reporting detection accuracy exceeding 99% on benchmark datasets [3], [4]. These results suggest that the problem of detecting known network attacks has been largely addressed.

However, these high accuracy numbers hide a critical limitation. Most existing deep learning IDS models operate under the closed-set assumption, meaning the model is trained only on attack types it has already seen and must assign every input to one of its known categories [5], [6]. When a new, previously unseen attack arrives (commonly referred to as a zero-day attack), the system has no mechanism to recognize it as unfamiliar [7], [8]. Instead, it assigns the traffic to the closest known category, which is almost always BENIGN or normal traffic, and allows it to pass through undetected [6], [7]. This is a serious vulnerability because real-world networks face new and evolving threats constantly [5], [8].

To address this gap, this research proposes the ABOSD framework, a deep learning-based intrusion detection system that can both classify known attacks accurately and identify when it encounters traffic it has never seen before [6], [7]. The system was trained on CICIDS2017 [3] and evaluated on two completely separate datasets, UNSW-NB15 [12] and NSL-KDD [13], neither of which shares any attack types with the training data.

II. LITERATURE REVIEW

Ten closely related research papers were reviewed to understand the current state of the field and identify exactly where the gap lies.

Outside the Closed World [5]: Sommer and Paxson formally proved that all machine learning IDS models operate under a closed-set assumption that does not hold in real-world environments where new attacks continuously appear. This paper is the founding work behind the problem this research addresses, and the fact that it remains unresolved after 15 years directly justifies the need for this study.

Adaptable Deep Learning-Based IDS to Zero-Day Attacks [6]: Soltani et al. attempted an open-set IDS using Deep Open Classification and OpenMax techniques on CICIDS2017, achieving 85-89% unknown attack detection. However, the system required complex clustering, expert labeling, and repeated retraining, making it impractical for real-world deployment.

From Zero-Shot ML to Zero-Day Attack Detection [7]: Sarhan et al. proposed using zero-shot learning so the model reasons about unseen attack characteristics. Evaluated on UNSW-NB15 and CICIDS2017, it achieved 78-84% zero-day detection and confirmed that the standard closed-set evaluation approach is fundamentally flawed.

CICIDS2017 Dataset [3]: Sharafaldin et al. introduced the CICIDS2017 dataset used as training data in this research. It contains 2.8 million records, 79 features, and 14 modern attack categories collected over five days of realistic network traffic.

UNSW-NB15 Dataset [12]: Moustafa and Slay introduced UNSW-NB15, used as the first unknown test dataset. Its attack categories, which include Fuzzers, Backdoors, Shellcode, Worms, and Exploits, do not appear in CICIDS2017 at all, making it a good choice for evaluating detection of unseen threats.

NSL-KDD Dataset [13]: Tavallaei et al. introduced the improved NSL-KDD dataset, used as the second unknown test dataset. Its attack types originate from 1999 and are absent from both CICIDS2017 and UNSW-NB15, allowing evaluation across a significant time gap.

Unknown Intrusion Detection Using Open-Set Recognition [8]: Nguyen et al. explained technically why SoftMax-based models fail on unknown attacks. Replacing SoftMax with a generative flow-based model improved unknown attack detection by 12%, providing strong technical justification that the problem lies in model architecture.

Zero-Day Attack Detection: A Survey [9]: Guo reviewed multiple studies and reported that while models achieve 95-99% accuracy on known attacks, detection of truly unknown attacks remains limited to 70-85%, directly highlighting the gap between reported academic performance and real-world capability.

Deep Learning for Cyber Security Intrusion Detection [10]: Ferrag et al. surveyed multiple architectures including CNN, RNN, LSTM, and autoencoders. None of the evaluated models were tested for detecting unknown attacks across different datasets, confirming that generalization has been consistently overlooked.

Network Intrusion Detection: A Systematic Study [11]: Ahmad et al. reviewed over 70 intrusion detection studies and found that despite high reported accuracy, real-world deployment remains limited due to unrealistic datasets, class imbalance, and lack of testing on unseen data.

Across all ten papers, the consistent finding is that standard deep learning IDS models perform well on attacks they have been trained on but fail silently on anything new [5], [8], [9], [10]. No existing work provides a simple, reproducible framework trained once and evaluated on two completely separate unknown datasets without any retraining. That is the exact gap this research fills.

III. CORE CONTRIBUTION: DEEP LEARNING FOR BIG DATA

This section addresses the critical interplay between the ABOSD framework and the Big Data characteristics inherent in large-scale network traffic datasets. Modern network environments generate data at a scale, velocity, and variety that renders traditional machine learning approaches fundamentally inadequate.

A. Scale and Volume

The CICIDS2017 dataset alone contains approximately 2.8 million network flow records across 79 numerical features, amounting to hundreds of gigabytes of raw pcap data. When combined with UNSW-NB15 (82,332 test records) and NSL-KDD (22,544 test records), the total dataset volume represents a non-trivial Big Data processing challenge that directly influenced every architectural decision in this research.

To handle this volume efficiently, the following techniques were employed. First, class-balanced downsampling was applied to CICIDS2017, reducing the dominant BENIGN class to three times the total attack count. This reduced training data to a manageable subset while preserving statistical representation of all attack categories. Second, the LSTM model uses gradient-based optimization (Adam) with mini-batch training, meaning the model never needs to hold the full 2.8M-record dataset in memory at once. Third, the autoencoder operates entirely on pre-scaled feature vectors (dimensionality: 79), making each forward pass computationally cheap regardless of dataset size, so inference across all 82,332 UNSW-NB15 flows can be done in a single batch.

The architectural choice of a fully connected autoencoder over more computationally expensive alternatives (e.g., convolutional or transformer-based autoencoders) was deliberate: for tabular network flow data at this scale, dense layers provide sufficient expressive power with substantially lower memory and compute overhead than attention-based or spatial architectures.

B. Velocity and Real-Time Detection Suitability

Network traffic is inherently a high-velocity data stream. Enterprise networks routinely process thousands of packets per second. The ABOSD framework was designed with this in mind. The autoencoder performs a single forward pass per incoming flow, which is basically a matrix multiply through six dense layers, and produces a scalar reconstruction error very quickly. This makes the system suitable for real-time deployment in streaming

network environments where decisions need to be made fast.

The threshold mechanism adds zero additional latency: a single scalar comparison ($\text{error} \geq \text{threshold}$) is performed after each autoencoder inference. This contrasts with competing approaches such as OpenMax [6] or generative flow-based models [8], which require iterative computations or large generative networks that are incompatible with low-latency deployment.

C. Variety: Heterogeneous Multi-Dataset Evaluation

Variety is the Big Data characteristic most often ignored in IDS research. Most published systems train and test on the same dataset, making performance metrics meaningless in practice [9], [10]. This research explicitly confronts variety by evaluating on three structurally different datasets generated by different tools, in different time periods (1999 to 2018), on different network topologies, with different feature extraction pipelines and completely non-overlapping attack taxonomies.

CICIDS2017 uses CICFlowMeter features (79 features). UNSW-NB15 uses IXIA PerfectStorm-generated traffic with different feature semantics (49 native features). NSL-KDD uses KDDCup99 features from 1999 traffic captures (41 native features). To handle this variety without retraining, zero-padding was applied to align all datasets to the common 79-feature input dimension. While this introduces noise, the reconstruction error gap between known and unknown traffic was so large (1,452x for UNSW-NB15) that the variety-induced noise did not prevent reliable detection.

D. Veracity: Handling Noisy and Imbalanced Big Data

Real-world network traffic datasets suffer from significant veracity issues including label noise, class imbalance, infinity values produced by feature extraction division errors, and NaN values from dropped flows. The preprocessing pipeline addressed these directly. All rows containing infinity or NaN values were removed prior to training. Class imbalance was addressed through stratified downsampling of BENIGN records, preventing the model from achieving high accuracy by predicting the majority class. MinMaxScaler normalization was fit exclusively on training data and applied identically to all three test datasets, preventing data leakage and ensuring consistent feature scaling across heterogeneous sources.

E. ETL Pipeline and Data Bottleneck Management

The Extract, Transform, Load (ETL) pipeline proceeded as follows. In the extraction phase, CICIDS2017 CSV files (split across multiple daily files) were loaded and concatenated using pandas with `low_memory=False` to prevent dtype inference errors on large files. UNSW-NB15 was loaded from a pre-converted Parquet format for faster columnar access. NSL-KDD was loaded from its raw text format with manual column naming.

In the transformation phase, column names were stripped of whitespace (a common artefact of CICFlowMeter output), labels were encoded using scikit-learn LabelEncoder, features were selected by data type (numerical only), and all datasets were aligned to the 79-feature input dimension using zero-padding. In the loading phase, an 80/20 stratified train-test split was applied to CICIDS2017, and the fitted scaler was serialized so the same normalization could be applied to all three datasets consistently during inference. This pipeline ensured reproducibility and prevented any form of information leakage from test datasets into the training process.

F. Why Deep Learning Over Traditional Methods at This Scale

Traditional machine learning approaches such as Random Forests, SVMs, and Decision Trees face several fundamental limitations at the scale of modern network traffic data. Random Forests require all training data to be loaded into memory to build ensemble trees, which becomes infeasible at 2.8M+ records with 79 features without distributed infrastructure. SVMs scale as $O(n^2)$ to $O(n^3)$ in training complexity, making them computationally prohibitive for millions of samples. Neither approach provides a natural mechanism for open-set detection without extensive post-hoc modifications.

Deep learning, and specifically the LSTM-autoencoder combination used in ABOSD, overcomes these limitations through mini-batch gradient descent (which keeps memory usage constant regardless of dataset size), learned feature representations that generalize across different network environments, and the autoencoder's built-in anomaly detection capability that comes from its reconstruction objective. This last property emerges naturally from the architecture without needing any extra engineering.

IV. METHODOLOGY AND MODEL ARCHITECTURE

A. Baseline Model: LSTM Classifier

The baseline model is a Long Short-Term Memory network (LSTM) [1], a type of recurrent neural network designed for sequential data. Network traffic is sequential in nature, since the pattern of packets over time is what distinguishes normal traffic from attacks. LSTM handles this by maintaining a memory cell that carries information across time steps. The architecture consists of two stacked LSTM layers (128 and 64 units), a Dense layer (32 neurons, ReLU), and a SoftMax output layer.

The SoftMax function is defined as:

$$P(y = k \mid x) = e^{(z_k)} / \sum_{j=1, \dots, K} e^{(z_j)},$$

where z_k is the raw score for class k and K is the total number of known classes [6]. SoftMax forces all class probabilities to sum to exactly 1.0, so the model will always assign 100% confidence across its known classes regardless of what input it gets. This makes it

mathematically incapable of expressing uncertainty on inputs it has never seen [7], [8].

B. Proposed System: ABOSD Framework

To address this limitation, this study proposes the Autoencoder-Based Open-Set Detection (ABOSD) framework. An autoencoder is an unsupervised neural network that learns to compress input data into a lower-dimensional representation and then reconstruct the original input from that compressed form [1], [6]. When trained exclusively on CICIDS2017 known traffic, the autoencoder develops an internal representation of familiar traffic patterns. When it encounters traffic from a different distribution, it fails to reconstruct accurately, producing a significantly higher Mean Squared Error (MSE):

$$MSE = (1/n) \sum_{i=1, \dots, n} (x_i - \hat{x}_i)^2,$$

where x_i is the original feature value and $\hat{x}_i$ is the reconstructed value [6]. The autoencoder architecture is detailed in Table II.

TABLE II: AUTOENCODER ARCHITECTURE

Component	Layer	Size
Encoder	Input → Dense (ReLU)	79 → 64
Encoder	Dense (ReLU)	64 → 32
Encoder	Dense (ReLU) Bottleneck	32 → 16
Decoder	Dense (ReLU)	16 → 32
Decoder	Dense (ReLU)	32 → 64
Decoder	Dense (Sigmoid) Output	64 → 79

C. Architecture Pipeline

Each incoming network flow is first passed through the autoencoder. If reconstruction error exceeds the threshold, the flow is labeled Suspicious and flagged for human review, regardless of what the LSTM predicts. If the error is below the threshold, the flow is passed to the LSTM classifier which assigns it to one of the known attack categories [6], [7].

D. Threshold Selection

All percentile values between the 50th and 98th of the reconstruction error on CICIDS2017 known traffic were evaluated. Each threshold was scored using:

$$Score = 0.7 \times Detection\ Rate + 0.3 \times Pass\ Rate$$

The 70% weight on detection rate reflects the security principle that missing a real attack is more serious than a false alarm [5]. The optimal threshold was identified at the 98th percentile of known traffic reconstruction error, with a value of 0.000083.

E. Datasets

Three datasets were used, each serving a specific purpose, as detailed in Table III. The complete separation of attack types across all three datasets ensures that the evaluation represents genuine unknown attack detection with no overlap.

TABLE III: DATASET PROFILE

Dataset	Records	Features	Classes / Attacks	Role in Study
CICIDS2017	~2.8M	79	14 (DoS, Brute Force, PortScan, Web Attack, Bot, etc.)	Training only
UNSW-NB15	~82,332	49 → 79*	9 (Fuzzers, Backdoor, Shellcode, Worms, Exploits, etc.)	Unknown test set 1
NSL-KDD	~22,544	41 → 79*	4 families (Neptune, Smurf, Satan, R2L, etc.)	Unknown test set 2

* Zero-padded to match 79-feature input dimension

F. Preprocessing Pipeline

All datasets were cleaned by removing rows containing infinity or null values produced during feature extraction [3]. Class imbalance in CICIDS2017 was addressed by downsampling BENIGN to three times the total attack count before splitting [3], [9]. An 80/20 train-test split was applied using stratified sampling. Features were scaled to [0,1] using MinMaxScaler fitted only on training data, and the same scaler was applied to all three datasets to prevent data leakage.

V. EXPERIMENTS AND RESULTS

A. Training on CICIDS2017

The LSTM classifier was trained on the balanced CICIDS2017 training split using Adam optimizer with early stopping (patience = 5 epochs). The autoencoder was trained separately in a self-supervised manner on the same training split. Since its input and target are the same, it just learns to reconstruct its own input, which is how it builds an understanding of what normal CICIDS2017 traffic looks like [1], [6].

B. Performance on Known Attacks

The baseline LSTM achieved 87.23% overall accuracy and a weighted F1 score of 0.8347 on the held-out CICIDS2017 test set. However, per-class analysis

reveals serious weaknesses: PortScan, Bot, Heartbleed, Infiltration, Web Attack XSS, and Web Attack SQL Injection all achieved 0% recall. The macro-average recall across all 15 classes was only 39%. The 87% overall number is misleading because it is dominated by the large BENIGN class, and residual class imbalance severely affected learning for the minority attack categories [3], [9].

C. Performance on Unknown Attacks

For the baseline LSTM, results confirmed the closed-set failure mode completely [5], [6], [7]. On UNSW-NB15, the model classified 81,945 out of 82,332 unknown attacks as BENIGN with 98.55% average confidence. Not a single flow was correctly flagged. On NSL-KDD, the same pattern held.

For the ABOSD framework, reconstruction error showed a clear and large separation between known and unknown traffic. Known CICIDS2017 traffic had a mean reconstruction error of 0.000028. UNSW-NB15 unknowns had a mean error of 0.040658, which is 1,452 times larger. NSL-KDD unknowns had a mean error of 0.010712, which is 382 times larger. This large gap is what allows the threshold to reliably separate known from unknown traffic.

D. Comparative Results

Table I presents the full comparison across all three approaches evaluated in this research.

TABLE I: COMPARATIVE RESULTS ACROSS ALL THREE MODELS

Metric	Baseline LSTM	Confidence Threshold	ABOSD (Ours)
Approach	SoftMax classifier	SoftMax + threshold	Autoencoder error
Suspicious output	No	Partial	Yes
Known attack accuracy	87.23%	87.41%	98.0% pass-through
Avg confidence on unknowns	98.55% (wrong)	98.80% (wrong)	N/A
UNSW-NB15 detected	0.00%	2.74%	99.85%
UNSW-NB15 missed	100%	97.26%	0.15%
NSL-KDD detected	0.00%	N/A	90.20%
NSL-KDD missed	100%	N/A	9.80%
False alarm rate	N/A	14.48%	2.00%
Safe for deployment	No	No	Yes

These results show three things clearly. First, the baseline LSTM completely fails on unknown attacks despite being

very confident [5], [6], [7]. Second, confidence thresholding did not work because the LSTM was actually more confident on unknown inputs (98.80%) than on known ones, so the threshold basically never fired and only 2.74% of unknowns were caught. Third, ABOSD detected 99.85% of UNSW-NB15 unknowns and 90.20% of NSL-KDD unknowns with only a 2% false alarm rate, which makes it much more suitable for real-world deployment [12], [13].

VI. DISCUSSION AND FUTURE WORK

The main limitations of this work include residual class imbalance affecting per-class recall on known attacks [3], [9], the use of zero-padding to match feature dimensions which may introduce noise, and the static nature of the threshold which does not adapt to evolving traffic over time [5]. Scaling to fully distributed training environments (e.g., Apache Spark ML, Horovod for distributed TensorFlow) would enable processing of terabyte-scale network captures from enterprise environments without the downsampling currently required.

Future work could explore adaptive thresholding mechanisms that update the detection boundary in response to concept drift, attention mechanisms within the autoencoder to weight anomalous features more heavily, transformer-based sequence models for capturing longer temporal dependencies in multi-packet flows, and continuous online learning to handle evolving network traffic patterns without periodic full retraining. Integration with real-time stream processing frameworks such as Apache Kafka or Apache Flink would extend the system to production-grade deployment at enterprise scale.

VII. CONCLUSION

This research addressed a well-known and persistently unsolved problem in network intrusion detection, which is the inability of standard deep learning classifiers to detect attack types they have never seen before [5], [8], [9], [10]. The baseline LSTM, representing the existing state of the art, classified every single unknown attack as normal traffic with near-perfect confidence [5], [6], [7]. The proposed ABOSD framework solved this by combining the LSTM with an autoencoder trained to recognize familiar traffic, using reconstruction error as the signal for flagging unfamiliar flows as suspicious [6], [8].

Trained once on CICIDS2017 [3] and tested on two completely different unknown datasets with no retraining, ABOSD detected 99.85% of UNSW-NB15 [12] unknowns and 90.20% of NSL-KDD [13] unknowns, with a false alarm rate of only 2%. The Core Contribution section showed that this approach directly addresses the volume, velocity, variety, and veracity challenges of Big Data network security, using architectural choices such as mini-batch LSTM training, lightweight autoencoder inference, and a single-pass threshold mechanism that are all compatible with real-time, high-throughput deployment environments.

REFERENCES

- [1] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.
- [2] M. Z. Alom et al., "A state-of-the-art survey on deep learning theory and architectures," *Electronics*, vol. 8, no. 3, p. 292, 2019.
- [3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSP*, 2018, pp. 108-116.
- [4] W. Wang et al., "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE ISSNIP*, 2017.
- [5] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 305-316.
- [6] M. Soltani et al., "An adaptable deep learning-based intrusion detection system to zero-day attacks," *J. Inf. Secur. Appl.*, vol. 76, p. 103516, 2023.
- [7] M. Sarhan et al., "From zero-shot machine learning to zero-day attack detection," *Int. J. Inf. Secur.*, vol. 22, pp. 1103-1116, 2023.
- [8] T. Nguyen et al., "Unknown intrusion traffic detection using unsupervised learning and open-set recognition," *Sci. Rep.*, vol. 15, May 2025.
- [9] Y. Guo, "Machine learning-based zero-day attack detection: A survey," *Computers and Communications*, 2023.
- [10] M. A. Ferrag et al., "Deep learning for cyber security intrusion detection," *J. Inf. Secur. Appl.*, vol. 50, p. 102419, 2020.
- [11] Z. Ahmad et al., "Network intrusion detection system: A systematic study," *Trans. Emerg. Telecommun. Technol.*, vol. 32, no. 1, p. e4150, 2021.
- [12] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. MilCIS*, 2015, pp. 1-6.
- [13] M. Tavallaee et al., "A detailed analysis of the KDD Cup 99 data set," in *Proc. IEEE CISDA*, 2009.